FINAL Unit3 FRQ - Topic order

1. SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.

- Assume that the classes listed in the Java Quick Reference have been imported where appropriate.
- Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
- In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

This question involves methods that distribute text across lines of an electronic sign. The electronic sign and the text to be displayed on it are represented by the `Sign` class. You will write the complete `Sign` class, which contains a constructor and two methods.

The `Sign` class constructor has two parameters. The first parameter is a `String` that contains the message to be displayed on the sign. The second parameter is an `int` that contains the *width* of each line of the sign. The width is the positive maximum number of characters that can be displayed on a single line of the sign.

A sign contains as many lines as are necessary to display the entire message. The message is split among the lines of the sign without regard to spaces or punctuation. Only the last line of the sign may contain fewer characters than the width indicated by the constructor parameter.

The following are examples of a message displayed on signs of different widths. Assume that in each example, the sign is declared with the width specified in the first column of the table and with the message `"Everything on sale, please come in"`, which contains 34 characters.

Width of the Sign	Sign Display

15	`Everything on s`
	`ale, please com`
	`e in`

17	`Everything on sal` `e, please come in`
40	`Everything on sale, please come in`

In addition to the constructor, the `Sign` class contains two methods.

The `numberOfLines` method returns an `int` representing the number of lines needed to display the text on the sign. In the previous examples, `numberOfLines` would return `3`, `2`, and `1`, respectively, for the sign widths shown in the table.

The `getLines` method returns a `String` containing the message broken into lines separated by semicolons (`;`) or returns `null` if the message is the empty string. The constructor parameter that contains the message to be displayed will not include any semicolons. As an example, in the first row of the preceding table, `getLines` would return `"Everything on s;ale, please com;e in"`. No semicolon should appear at the end of the `String` returned by `getLines`.

The following table contains a sample code execution sequence and the corresponding results. The code execution sequence appears in a class other than `Sign`.

FINAL Unit3 FRQ - Topic order

Statement	Method Call Return Value (blank if none)	Explanation
`String str;`		
`int x;`		
`Sign sign1 = new Sign("ABC222DE", 3);`		The message for `sign1` contains 8 characters, and the sign has lines of width 3.
`x = sign1.numberOfLines();`	`3`	The sign needs three lines to display the 8-character message on a sign with lines of width 3.
`str = sign1.getLines();`	`"ABC;222;DE"`	Semicolons separate the text displayed on the first, second, and third lines of the sign.
`str = sign1.getLines();`	`"ABC;222;DE"`	Successive calls to `getLines` return the same value.
`Sign sign2 = new Sign("ABCD", 10);`		The message for `sign2` contains 4 characters, and the sign has lines of width 10.
`x = sign2.numberOfLines();`	`1`	The sign needs one line to display the 4-character message on a sign with lines of width 10.
`str = sign2.getLines();`	`"ABCD"`	No semicolon appears, since the text to be displayed fits on the first line of the sign.
`Sign sign3 = new Sign("ABCDEF", 6);`		The message for `sign3` contains 6 characters, and the sign has lines of width 6.
`x = sign3.numberOfLines();`	`1`	The sign needs one line to display the 6-character message on a sign with lines of width 6.
`str = sign3.getLines();`	`"ABCDEF"`	No semicolon appears, since the text to be displayed fits on the first line of the sign.
`Sign sign4 = new Sign("", 4);`		The message for `sign4` is an empty string.
`x = sign4.numberOfLines();`	`0`	There is no text to display.

FINAL Unit3 FRQ - Topic order

Statement	Method Call Return Value (blank if none)	Explanation
`str = sign4.getLines();`	`null`	There is no text to display.
`Sign sign5 = new Sign("AB_CD_EF", 2);`		The message for `sign5` contains 8 characters, and the sign has lines of width 2.
`x = sign5.numberOfLines();`	`4`	The sign needs four lines to display the 8-character message on a sign with lines of width 2.
`str = sign5.getLines();`	`"AB;_C;D_;EF"`	Semicolons separate the text displayed on the four lines of the sign.

Write the complete `Sign` class. Your implementation must meet all specifications and conform to the examples shown in the preceding table.

Sign class (9 points)

Select a point value to view scoring criteria, solutions, and/or examples and to score the response. +1 indicates a point earned and −1 indicates a general or question-specific penalty.

+1 **[Skill 3.B]** Declares class header: class Sign

Responses will not earn the point if they

- declare the class as something other than public

+1 **[Skill 3.B]** Declares appropriate private instance variable(s) and constructor initializes instance variables using appropriate parameters

Responses can still earn the point even if they

- store calculated values instead of the message and width, as long as the declared instance variables can collectively answer the questions and their values are computed from the parameters (correctly or incorrectly)

Responses will not earn the point if they

- declare the variable outside the class, or in the class within a method or constructor

+1 **[Skill 3.B]** Declares constructor header: Sign (String ___, int ___)

Responses can still earn the point even if they

FINAL Unit3 FRQ - Topic order

- calculate values in the constructor that are returned by other methods, correctly or incorrectly, as long as the parameter types are correct

Responses will not earn the point if they

- declare the constructor as something other than public

+1 **[Skill 3.B]** Declares method headers:

public int numberOfLines( )

public String getLines( )

Responses will not earn the point if they

- omit either method
- omit public or declare the method as something other than public

+1 **[Skill 3.C]** numberOfLines divides the message length by the line width

Responses can still earn the point even if they

- perform the division in a method other than numberOfLines
- perform the division without using the division operator by counting line-width-sized portions of the message or by counting lines produced by the line-delimiting algorithm
- incorrectly account for the final line
- use a method name inconsistent with the examples, as long as it is recognizably equivalent

+1 **[Skill 3.C]** numberOfLines returns appropriate value (*algorithm*)

Responses can still earn the point even if they

- perform the return value calculation in the constructor
- return a different number of lines than getLines produces, as long as the number returned is the correct number for the message
- return an incorrect number of lines for the message, as long as the number returned is exactly the number of lines produced by getLines
- use a method name inconsistent with the examples, as long as it is recognizably equivalent

Responses will not earn the point if they

- incorrectly account for the final line

FINAL Unit3 FRQ - Topic order

+1 **[Skill 3.C]** getLines returns null appropriately

Responses can still earn the point even if they

- identify null case in a method other than getLines
- use an invalid call to length or == in guard for null return
- use a method name inconsistent with the examples, as long as it is recognizably equivalent

Responses will not earn the point if they

- guard the return with incorrect logic

+1 **[Skill 3.A]** Calls substring and length (or equivalent) on String objects

Responses can still earn the point even if they

- calculate substring parameter values incorrectly
- call substring and/or length from a method other than getLines
- use a method name inconsistent with the examples, as long as it is recognizably equivalent

Responses will not earn the point if they

- fail to call substring or length on String objects
- call substring or length with an incorrect number of parameters, with a parameter of an incorrect type, or with incorrectly ordered parameters, anywhere in the class

+1 **[Skill 3.C]** getLines constructs the delimited sign output appropriately (*algorithm*)

Responses can still earn the point even if they

- call substring and/or length incorrectly
- fail to return the constructed String (*return not assessed*)
- handle the empty string/null case incorrectly
- construct the output in the constructor
- use a method name inconsistent with the examples, as long as it is recognizably equivalent

Responses will not earn the point if they

- end the constructed output with a ; or extraneous spaces
- modify the contents of message or width after they have been initialized

FINAL Unit3 FRQ - Topic order

Canonical Solution:

```
public class Sign
{
   private String message;
   private int width;

   public Sign(String m, int w)
   {
      message = m;
      width = w;
   }

   public int numberOfLines()
   {
      int len = message.length();
      if (len % width == 0)
      {
         return len / width;
      }
      else
      {
         return len / width + 1;
      }
   }

   public String getLines()
   {
      int linesNeeded = numberOfLines();
      if (linesNeeded  == 0)
      {
         return null;
      }

      String signLines = "";
      for (int i = 1; i < linesNeeded; i++)
      {
         signLines += message.substring((i - 1) *
                      width, i * width) + ";";
      }
      return signLines +
         message.substring((linesNeeded - 1) *
         width);
   }
}
```

FINAL Unit3 FRQ - Topic order

0	1	2	3	4	5	6	7	8	9 ✓

Total number of points earned (minus penalties) is equal to 9.

- ☐ +1 Declares class header: class Sign **(Points Earned)**
- ☐ +1 Declares appropriate private instance variable(s) and constructor initializes instance variables using appropriate parameters **(Points Earned)**
- ☐ +1 Declares constructor header: Sign (String ___, int ___) **(Points Earned)**
- ☐ +1 Declares method headers: **(Points Earned)**
  public int numberOfLines( )
  public String getLines( )
- ☐ +1 numberOfLines divides the message length by the line width **(Points Earned)**
- ☐ +1 numberOfLines returns appropriate value (*algorithm*) **(Points Earned)**
- ☐ +1 getLines returns null appropriately **(Points Earned)**
- ☐ +1 Calls substring and length (or equivalent) on String objects **(Points Earned)**
- ☐ +1 getLines constructs the delimited sign output appropriately (*algorithm*) **(Points Earned)**

Canonical Solution:

FINAL Unit3 FRQ - Topic order

```
public class Sign
{
   private String message;
   private int width;

   public Sign(String m, int w)
   {
      message = m;
      width = w;
   }

   public int numberOfLines()
   {
      int len = message.length();
      if (len % width == 0)
      {
         return len / width;
      }
      else
      {
         return len / width + 1;
      }
   }

   public String getLines()
   {
      int linesNeeded = numberOfLines();
      if (linesNeeded  == 0)
      {
         return null;
      }

      String signLines = "";
      for (int i = 1; i < linesNeeded; i++)
      {
         signLines += message.substring((i - 1) *
                      width, i * width) + ";";
      }
      return signLines +
         message.substring((linesNeeded - 1) *
         width);
   }
}
```

FINAL Unit3 FRQ - Topic order

2. The Car class, which you will write, is used to represent information about a car.

Car objects are created by calls to a constructor with two double parameters. The first parameter represents the number of gallons of fuel in the car. The second parameter represents the average number of miles the car can travel per gallon of fuel.

The Car class contains a canMakeTrip method, which determines whether there is enough fuel in the car to make a trip. This method has a double parameter that represents the distance, in miles, of a potential trip. If there is enough fuel, the method updates the current amount of fuel in the car and returns true. If there is not enough fuel for the entire distance of the trip, the amount of fuel is unchanged and the method returns false.

The following table contains a sample code execution sequence and the corresponding results. The code execution sequence appears in a class other than Car.

Statement	Return Value (blank if no value)	Explanation
boolean possible;		
Car carOne = new Car(10.0, 30.0);		A Car object named CarOne is constructed with 10 gallons of fuel and an average of 30 miles per gallon.
possible = carOne.canMakeTrip(120.0);	true	At 30 miles per gallon, a 120-mile trip will require 4 gallons of fuel. Since this is less than or equal to the 10 gallons of fuel remaining, the trip can be made. The amount of fuel remaining will be 6 gallons.
possible = carOne.canMakeTrip(90.0);	true	At 30 miles per gallon, a 90-mile trip will require 3 gallons of fuel. Since this is less than or equal to the 6 gallons of fuel remaining, the trip can be made. The amount of fuel remaining will be 3 gallons.

FINAL Unit3 FRQ - Topic order

Statement	Return Value (blank if no value)	Explanation
possible = carOne.canMakeTrip(225.0);	false	At 30 miles per gallon, a 225-mile trip will require 7.5 gallons of fuel. Since this is more than the 3 gallons of fuel remaining, the trip cannot be made and the amount of fuel is unchanged.
Car carTwo = new Car(12.0, 15.0);		A Car object named carTwo is constructed with 12 gallons of fuel and an average of 15 miles per gallon.
possible = carTwo.canMakeTrip(300.0);	false	At 15 miles per gallon, a 300-mile trip will require 20 gallons of fuel. Since this is more than the 12 gallons of fuel remaining, the trip cannot be made and the amount of fuel is unchanged.
possible = carTwo.canMakeTrip(180.0);	true	At 15 miles per gallon, a 180-mile trip will require 12 gallons of fuel. Since this is less than or equal to the 12 gallons of fuel remaining, the trip can be made. The amount of fuel remaining will be 0 gallons.

Write the complete `Car` class. Your implementation must meet all specifications and conform to the examples shown in the table.

Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares class header: `class Car`

Canonical Solution:

```
public class Car
{
   private double currentFuel;
```

FINAL Unit3 FRQ - Topic order

```
   private double mpg;

   public Car(double c, double m)
   {
      currentFuel = c;
      mpg = m;
   }

   public boolean canMakeTrip(double distance)
   {
      double fuelRequired = distance / mpg;
      if (fuelRequired <= currentFuel)
      {
         currentFuel -= fuelRequired;
         return true;
      }
      else
      {
         return false;
      }
   }
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- declare the class as `private`
- include extraneous code outside the class
- Include () with or without parameters in the class header

Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares `private` instance variables for the current fuel and miles per gallon

Canonical Solution:

FINAL Unit3 FRQ - Topic order

```
public class Car
{
   private double currentFuel;
   private double mpg;

   public Car(double c, double m)
   {
      currentFuel = c;
      mpg = m;
   }

   public boolean canMakeTrip(double distance)
   {
      double fuelRequired = distance / mpg;
      if (fuelRequired <= currentFuel)
      {
         currentFuel -= fuelRequired;
         return true;
      }
      else
      {
         return false;
      }
   }
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point if they

- incorrectly initialize the instance variables

Responses will not earn the point if they

- declare any instance variable `static`
- omit `private` in instance variable declaration
- declare a variable outside the class
- declare an instance variable inside the constructor

Point 3

FINAL Unit3 FRQ - Topic order

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares constructor header: `Car(double ___, double ___)`

Canonical Solution:

```
public class Car
{
   private double currentFuel;
   private double mpg;

   public Car(double c, double m)
   {
      currentFuel = c;
      mpg = m;
   }

   public boolean canMakeTrip(double distance)
   {
      double fuelRequired = distance / mpg;
      if (fuelRequired <= currentFuel)
      {
         currentFuel -= fuelRequired;
         return true;
      }
      else
      {
         return false;
      }
   }
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- declare the constructor as `private` or `static`

Point 4

FINAL Unit3 FRQ - Topic order

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Assigns correct initial values to all instance variables

Canonical Solution:

```
public class Car
{
   private double currentFuel;
   private double mpg;

   public Car(double c, double m)
   {
      currentFuel = c;
      mpg = m;
   }

   public boolean canMakeTrip(double distance)
   {
      double fuelRequired = distance / mpg;
      if (fuelRequired <= currentFuel)
      {
         currentFuel -= fuelRequired;
         return true;
      }
      else
      {
         return false;
      }
   }
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point if they

- write a class header with parameter and assign the parameter to the instance variables

Responses will not earn the point if they

FINAL Unit3 FRQ - Topic order

- fail to declare or initialize instance variable for capacity
- assign values to instance variables from an unknown source
- assigns initial value to local variable instead of instance variable, even if the local variables are declared as `private`

Point 5

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares method header: `boolean canMakeTrip(double ___)`

Canonical Solution:

```
public class Car
{
   private double currentFuel;
   private double mpg;

   public Car(double c, double m)
   {
      currentFuel = c;
      mpg = m;
   }

   public boolean canMakeTrip(double distance)
   {
      double fuelRequired = distance / mpg;
      if (fuelRequired <= currentFuel)
      {
         currentFuel -= fuelRequired;
         return true;
      }
      else
      {
         return false;
      }
   }
}
```



0	1

FINAL Unit3 FRQ - Topic order

The response earns the point.

Responses will not earn the point if they

- declare method as something other than `public`
- use incorrect method name
- use incorrect return or parameter type

Point 6

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Determines whether there is enough fuel for the trip and updates fuel when appropriate (*algorithm*)

Canonical Solution:

```
public class Car
{
   private double currentFuel;
   private double mpg;

   public Car(double c, double m)
   {
      currentFuel = c;
      mpg = m;
   }

   public boolean canMakeTrip(double distance)
   {
      double fuelRequired = distance / mpg;
      if (fuelRequired <= currentFuel)
      {
         currentFuel -= fuelRequired;
         return true;
      }
      else
      {
         return false;
      }
   }
}
```

FINAL Unit3 FRQ - Topic order

	✓
0	1

The response earns the point.

Responses can still earn the point if they

- fail to return the `boolean` value
- use a variable declared outside the method as an instance variable when instance variables do not exist
- declare instance variables outside the class, or in the class within a method or constructor

Responses will not earn the point if they

- fail to update the instance variable(s)
- determine the correct result but return something else

Point 7

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Returns determined `boolean` value for whether the trip can be made

Canonical Solution:

```
public class Car
{
   private double currentFuel;
   private double mpg;

   public Car(double c, double m)
   {
      currentFuel = c;
      mpg = m;
   }

   public boolean canMakeTrip(double distance)
   {
      double fuelRequired = distance / mpg;
      if (fuelRequired <= currentFuel)
      {
         currentFuel -= fuelRequired;
         return true;
```

FINAL Unit3 FRQ - Topic order

```
      }
      else
      {
         return false;
      }
   }
}
```

	✓
0	1

The response earns the point.

Responses can still earn the point if they

- return an incorrect value

Responses will not earn the point if they

- fail to return a value in some cases

FINAL Unit3 FRQ - Topic order

3. The Cube class represents cubes that are used in a board game. There are three types of cubes.

- Number cubes, whose faces are labeled with the numbers 1 through 6
- Letter cubes, whose faces are labeled with the letters A through F
- Color cubes, whose faces are labeled with the colors red, orange, yellow, green, blue, and purple

A partial Cube class definition is shown.

```
public class Cube
{
  /**
   *  Constructs a Cube object. If the parameter is
   *  equal to "number", the cube will be a number
   *  cube. If the parameter is equal to "letter", the
   *  cube will be a letter cube. If the parameter is
   *  equal to "color", the cube will be a color cube.
   *  Precondition: cubeType is equal to "number",
   *                "letter", or "color".
   */
  public Cube(String cubeType)
  { /* implementation not shown */ }

  /**
   *  Returns "1", "2", "3", "4", "5", or "6" if
   *  the cube is a number cube, returns "A", "B",
   *  "C", "D", "E", or "F" if the cube is a
   *  letter cube, and returns "red", "orange",
   *  "yellow", "green", "blue", or "purple" if
   *  the cube is a color cube
   */
  public String roll()
  { /* implementation not shown */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

You will write a class CubePair to represent two Cube objects that are used in a game. The CubePair class should have a one-parameter constructor, a two-parameter constructor, and a rollCubes method.

The following table contains a sample code execution sequence and the corresponding results. The code execution sequence appears in a class other than Cube or CubePair.

FINAL Unit3 FRQ - Topic order

Statement	Possible Return Value (blank if no value)	Explanation
CubePair p1 = new CubePair("number", "color");		Constructs a CubePair object with two Cube attributes: one number cube and one color cube.
p1.rollCubes();	"2yellow"	Returns a String consisting of the results obtained by rolling the two cubes associated with p1.
CubePair p2 = new CubePair("number");		Constructs a CubePair object with two Cube attributes that are both number cubes.
p2.rollCubes();	"53"	Returns a String consisting of the results obtained by rolling the two cubes associated with p2.
CubePair p3 = new CubePair("letter");		Constructs a CubePair object with two Cube attributes that are both letter cubes.
p3.rollCubes();	"DC"	Returns a String consisting of the results obtained by rolling the two cubes associated with p3.

Assume there is a precondition that all arguments passed to a CubePair constructor are equal to "number", "letter", or "color".

Write the complete CubePair class. Your implementation must meet all specifications and conform to the examples shown in the table.

Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares class header: `class CubePair`

Canonical Solution:

FINAL Unit3 FRQ - Topic order

```
public class CubePair
{
   private Cube cube1;
   private Cube cube2;

   public CubePair(String type)
   {
      cube1 = new Cube(type);
      cube2 = new Cube(type);
   }

   public CubePair(String type1, String type2)
   {
      cube1 = new Cube(type1);
      cube2 = new Cube(type2);
   }

   public String rollCubes()
   {
      String result = "";
      result += cube1.roll();
      result += cube2.roll();
      return result;
   }
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- declare the class as `private`
- include extraneous code outside the class

Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares two `private Cube` instance variables to represent each cube

FINAL Unit3 FRQ - Topic order

Canonical Solution:

```
public class CubePair
{
   private Cube cube1;
   private Cube cube2;

   public CubePair(String type)
   {
      cube1 = new Cube(type);
      cube2 = new Cube(type);
   }

   public CubePair(String type1, String type2)
   {
      cube1 = new Cube(type1);
      cube2 = new Cube(type2);
   }

   public String rollCubes()
   {
      String result = "";
      result += cube1.roll();
      result += cube2.roll();
      return result;
   }
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- declare any instance variable `static`
- declare a variable outside the class

Point 3

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

FINAL Unit3 FRQ - Topic order

- Declares two constructor headers: `CubePair(String ___)` and `CubePair(String ___, String ___)`

Canonical Solution:

```
public class CubePair
{
   private Cube cube1;
   private Cube cube2;

   public CubePair(String type)
   {
      cube1 = new Cube(type);
      cube2 = new Cube(type);
   }

   public CubePair(String type1, String type2)
   {
      cube1 = new Cube(type1);
      cube2 = new Cube(type2);
   }

   public String rollCubes()
   {
      String result = "";
      result += cube1.roll();
      result += cube2.roll();
      return result;
   }
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- declare either constructor as `private`

Point 4

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

FINAL Unit3 FRQ - Topic order

- Constructor assigns correct initial values to all instance variables

Canonical Solution:

```
public class CubePair
{
   private Cube cube1;
   private Cube cube2;

   public CubePair(String type)
   {
      cube1 = new Cube(type);
      cube2 = new Cube(type);
   }

   public CubePair(String type1, String type2)
   {
      cube1 = new Cube(type1);
      cube2 = new Cube(type2);
   }

   public String rollCubes()
   {
      String result = "";
      result += cube1.roll();
      result += cube2.roll();
      return result;
   }
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- fail to declare or initialize instance variables for the cubes
- make any invalid call to the `Cube` constructor within the `CubePair` class

Point 5

Select a point value to view scoring criteria or to score the response.

FINAL Unit3 FRQ - Topic order

To earn this point, responses must demonstrate the following:

- Declares method header: `String rollCubes()`

Canonical Solution:

```
public class CubePair
{
   private Cube cube1;
   private Cube cube2;

   public CubePair(String type)
   {
      cube1 = new Cube(type);
      cube2 = new Cube(type);
   }

   public CubePair(String type1, String type2)
   {
      cube1 = new Cube(type1);
      cube2 = new Cube(type2);
   }

   public String rollCubes()
   {
      String result = "";
      result += cube1.roll();
      result += cube2.roll();
      return result;
   }
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- use incorrect method name
- declare method as something other than `public`

Point 6

FINAL Unit3 FRQ - Topic order

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Method calls `roll` on each cube and concatenates result (*algorithm*)

Canonical Solution:

```
public class CubePair
{
   private Cube cube1;
   private Cube cube2;

   public CubePair(String type)
   {
      cube1 = new Cube(type);
      cube2 = new Cube(type);
   }

   public CubePair(String type1, String type2)
   {
      cube1 = new Cube(type1);
      cube2 = new Cube(type2);
   }

   public String rollCubes()
   {
      String result = "";
      result += cube1.roll();
      result += cube2.roll();
      return result;
   }
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- fail to return the string representing the two rolls
- declare instance variables outside the class, or in the class within a method or constructor
- construct new cubes for each roll, as long as they are of the appropriate cube type

FINAL Unit3 FRQ - Topic order

Point 7

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Method returns concatenated string

Canonical Solution:

```
public class CubePair
{
   private Cube cube1;
   private Cube cube2;

   public CubePair(String type)
   {
      cube1 = new Cube(type);
      cube2 = new Cube(type);
   }

   public CubePair(String type1, String type2)
   {
      cube1 = new Cube(type1);
      cube2 = new Cube(type2);
   }

   public String rollCubes()
   {
      String result = "";
      result += cube1.roll();
      result += cube2.roll();
      return result;
   }
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- return an incorrect string

FINAL Unit3 FRQ - Topic order

Responses will not earn the point if they

- return a single `String` literal with no computation
- return a non-`String` value
- fail to return a value in some cases

FINAL Unit3 FRQ - Topic order

4. The CupcakeMachine class, which you will write, represents a cupcake vending machine, an automated machine that dispenses cupcakes. CupcakeMachine objects are created by calls to a constructor with two parameters.

- The first parameter is an int that represents the number of cupcakes that the vending machine has been stocked with. Assume that this value will be greater than or equal to 0.
- The second parameter is a double that represents the cost, in dollars, per cupcake. Assume that this value will be greater than 0.0.

The CupcakeMachine class contains a takeOrder method, which determines whether a cupcake order can be filled. A cupcake order can be filled if there are at least as many cupcakes in the vending machine as there are in the order.

A cupcake order is represented by a single int parameter to the takeOrder method. Assume that all values passed to the takeOrder method are positive.

If an order can be filled, the method updates the number of cupcakes remaining in the machine and returns a String containing information about the order. The returned string indicates the order number and the cost of the order, as shown in the following table. Order numbers begin at 1 and increase by 1 for each order filled.

If the order cannot be filled because the vending machine does not have enough cupcakes, the takeOrder method should return the message "Order cannot be filled". In this case, the number of cupcakes available in the machine is unchanged and no order number is given to the order.

The following table contains a sample code execution sequence and the corresponding results. The code execution sequence appears in a class other than CupcakeMachine.

Statement	Return Value (blank if no value)	Explanation
String info;		
CupcakeMachine c1 = new CupcakeMachine(10, 1.75);		CupcakeMachine c1 is constructed with 10 cupcakes and a cost of $1.75 per cupcake.
info = c1.takeOrder(2);	"Order number 1, cost $3.5"	A customer orders 2 cupcakes for a total cost of 2 × $1.75.

FINAL Unit3 FRQ - Topic order

Statement	Return Value (blank if no value)	Explanation
		There are now 8 cupcakes remaining in the machine.
info = c1.takeOrder(3);	"Order number 2, cost $5.25"	A customer orders 3 cupcakes for a total cost of 3 × $1.75. There are now 5 cupcakes remaining in the machine.
info = c1.takeOrder(10);	"Order cannot be filled"	A customer attempts to order 10 cupcakes. The machine only has 5 cupcakes available, so no order is made.
info = c1.takeOrder(1);	"Order number 3, cost $1.75"	A customer orders 1 cupcake for a total cost of $1.75. There are now 4 cupcakes remaining in the machine.
CupcakeMachine c2 = new CupcakeMachine(10, 1.5);		CupcakeMachine c2 is constructed with 10 cupcakes and a cost of $1.50 per cupcake.
info = c2.takeOrder(10);	"Order number 1, cost $15.0"	A customer orders 10 cupcakes for a total cost of 10 × $1.50. There are now 0 cupcakes remaining in the machine.

Write the complete CupcakeMachine class. Your implementation must meet all specifications and conform to the examples shown in the table.

Point 1

Select a point value to view scoring criteria or to score the response.

FINAL Unit3 FRQ - Topic order

To earn this point, responses must demonstrate the following:

- Declares class header: `class CupcakeMachine`

Canonical Solution:

```
public class CupcakeMachine
{
   private int availCupcakes;
   private double cupcakeCost;
   private int orderNum;

   public CupcakeMachine(int num, double cost)
   {
      availCupcakes = num;
      cupcakeCost = cost;
      orderNum = 1;
   }

   public String takeOrder(int quantity)
   {
      String message = "Order cannot be filled";

      if (quantity <= availCupcakes)
      {
         availCupcakes -= quantity;
         double cost = quantity * cupcakeCost;
         message = "Order number " + orderNum
                   + ", cost $" + cost;
         orderNum++;
      }

      return message;
   }
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- declare the class as `private`

FINAL Unit3 FRQ - Topic order

- include extraneous code outside the class

Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares `private` instance variables of appropriate types to store the number of cupcakes, the order number, and the cost per cupcake

Canonical Solution:

```
public class CupcakeMachine
{
   private int availCupcakes;
   private double cupcakeCost;
   private int orderNum;

   public CupcakeMachine(int num, double cost)
   {
      availCupcakes = num;
      cupcakeCost = cost;
      orderNum = 1;
   }

   public String takeOrder(int quantity)
   {
      String message = "Order cannot be filled";

      if (quantity <= availCupcakes)
      {
         availCupcakes -= quantity;
         double cost = quantity * cupcakeCost;
         message = "Order number " + orderNum
                   + ", cost $" + cost;
         orderNum++;
      }

      return message;
   }
}
```

FINAL Unit3 FRQ - Topic order

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- declare any instance variable `static`
- declare a variable outside the class

Point 3

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares constructor header: `CupcakeMachine(int ___, double ___)`

Canonical Solution:

```
public class CupcakeMachine
{
   private int availCupcakes;
   private double cupcakeCost;
   private int orderNum;

   public CupcakeMachine(int num, double cost)
   {
      availCupcakes = num;
      cupcakeCost = cost;
      orderNum = 1;
   }

   public String takeOrder(int quantity)
   {
      String message = "Order cannot be filled";

      if (quantity <= availCupcakes)
      {
         availCupcakes -= quantity;
         double cost = quantity * cupcakeCost;
         message = "Order number " + orderNum
                   + ", cost $" + cost;
         orderNum++;
```

FINAL Unit3 FRQ - Topic order

```
      }

      return message;
   }
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- declare the constructor as `private`

Point 4

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Constructor assigns correct initial values to all instance variables

Canonical Solution:

```
public class CupcakeMachine
{
   private int availCupcakes;
   private double cupcakeCost;
   private int orderNum;

   public CupcakeMachine(int num, double cost)
   {
      availCupcakes = num;
      cupcakeCost = cost;
      orderNum = 1;
   }

   public String takeOrder(int quantity)
   {
      String message = "Order cannot be filled";

      if (quantity <= availCupcakes)
      {
```

FINAL Unit3 FRQ - Topic order

```
         availCupcakes -= quantity;
         double cost = quantity * cupcakeCost;
         message = "Order number " + orderNum
                   + ", cost $" + cost;
         orderNum++;
      }

      return message;
   }
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- initially assign an incorrect order number, as long as the algorithm corrects the order number before the first string is built

Responses will not earn the point if they

- fail to declare instance variables for number of cupcakes, order number, and cost per cupcake

Point 5

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares method header: `String takeOrder(int ___)`

Canonical Solution:

```
public class CupcakeMachine
{
   private int availCupcakes;
   private double cupcakeCost;
   private int orderNum;

   public CupcakeMachine(int num, double cost)
   {
      availCupcakes = num;
      cupcakeCost = cost;
      orderNum = 1;
```

FINAL Unit3 FRQ - Topic order

```
   }

   public String takeOrder(int quantity)
   {
      String message = "Order cannot be filled";

      if (quantity <= availCupcakes)
      {
         availCupcakes -= quantity;
         double cost = quantity * cupcakeCost;
         message = "Order number " + orderNum
                   + ", cost $" + cost;
         orderNum++;
      }

      return message;
   }
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- use incorrect method name
- declare method as something other than `public`

Point 6

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Builds message based on `int` parameter, updates number of available cupcakes, and updates the order number appropriately (*algorithm*)

Canonical Solution:

```
public class CupcakeMachine
{
   private int availCupcakes;
   private double cupcakeCost;
```

FINAL Unit3 FRQ - Topic order

```
   private int orderNum;

   public CupcakeMachine(int num, double cost)
   {
      availCupcakes = num;
      cupcakeCost = cost;
      orderNum = 1;
   }

   public String takeOrder(int quantity)
   {
      String message = "Order cannot be filled";

      if (quantity <= availCupcakes)
      {
         availCupcakes -= quantity;
         double cost = quantity * cupcakeCost;
         message = "Order number " + orderNum
                   + ", cost $" + cost;
         orderNum++;
      }

      return message;
   }
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- fail to return the message
- make minor spelling errors, as long as both `String` literal parts are included
- declare instance variables outside the class, or in the class within a method or constructor

Responses will not earn the point if they

- fail to handle the case when an order cannot be filled
- build correct messages for cases when an order can and cannot be filled, but return something else

Point 7

FINAL Unit3 FRQ - Topic order

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Returns appropriate constructed string message

Canonical Solution:

```
public class CupcakeMachine
{
   private int availCupcakes;
   private double cupcakeCost;
   private int orderNum;

   public CupcakeMachine(int num, double cost)
   {
      availCupcakes = num;
      cupcakeCost = cost;
      orderNum = 1;
   }

   public String takeOrder(int quantity)
   {
      String message = "Order cannot be filled";

      if (quantity <= availCupcakes)
      {
         availCupcakes -= quantity;
         double cost = quantity * cupcakeCost;
         message = "Order number " + orderNum
                   + ", cost $" + cost;
         orderNum++;
      }

      return message;
   }
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

FINAL Unit3 FRQ - Topic order

- return an incorrect message

Responses will not earn the point if they

- return a single `String` literal with no computation
- return a non-`String` value
- fail to return a value in some cases

FINAL Unit3 FRQ - Topic order

5. The LapTracker class, which you will write, is is used to count the number of laps a person walks around a track.

LapTracker objects are created by calls to a constructor with an int parameter that indicates how often to reset the LapTracker, with a precondition that the parameter is always positive. Newly constructed LapTracker objects start with a lap count of 0.

The LapTracker class contains an addLaps method, which adds laps to the lap count. This method has an int parameter that represents the number of laps to be added, with a precondition that the parameter is always positive. A LapTracker object repeatedly resets its lap count to 0 after a number of calls to addLaps, as indicated by its constructor parameter. The addLaps method returns an int that represents the lap count, as illustrated in the following table.

The table contains a sample code execution sequence and the corresponding results. The code execution sequence appears in a class other than LapTracker.

Statement	Return Value (blank if no value)	Explanation
int count;		
LapTracker amy = new LapTracker(3);		A LapTracker object named amy is constructed with an initial lap count of 0. The lap count will reset to 0 after every third call to amy's addLaps method.
count = amy.addLaps(8);	8	8 laps are added to the total lap count for amy.
count = amy.addLaps(12);	20	12 laps are added to the total lap count for amy.
count = amy.addLaps(10);	30	10 laps are added to the total lap count for amy. The addLaps method has been called on amy three times, so the next time the lap count will restart from 0.

FINAL Unit3 FRQ - Topic order

Statement	Return Value (blank if no value)	Explanation
count = amy.addLaps(11);	11	After the reset, 11 laps are added to the total lap count for amy.
count = amy.addLaps(5);	16	5 laps are added to the total lap count for amy.
count = amy.addLaps(13);	29	13 laps are added to the total lap count for amy. The addLaps method has been called on amy three more times, so the next time the lap count will restart from 0.
count = amy.addLaps(7);	7	After the reset, 7 laps are added to the total lap count for amy.
LapTracker eve = new LapTracker(2);		A LapTracker object named eve is constructed with an initial lap count of 0. The lap count will reset to 0 after every second call to eve's addLaps method.
count = eve.addLaps(20);	20	20 laps are added to the total lap count for eve.
count = eve.addLaps(13);	33	13 laps are added to the total lap count for eve. The addLaps method has been called on eve two times, so the next time the lap count will restart from 0.
count = eve.addLaps(10);	10	After the reset, 10 laps are added to the total lap count for eve.

Write the complete LapTracker class. Your implementation must meet all specifications and conform to the examples shown in the table.

Point 1

FINAL Unit3 FRQ - Topic order

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares class header: `class LapTracker`

Canonical Solution:

```
public class LapTracker
{
   private int numLaps;
   private int numCalls;
   private int resetFrequency;

   public LapTracker(int resetFreq)
   {
      numLaps = 0;
      numCalls = 0;
      resetFrequency = resetFreq;
   }

   public int addLaps(int additionalLaps)
   {
      if (numCalls == resetFrequency)
      {
         numCalls = 0;
         numLaps = 0;
      }
      numCalls++;
      numLaps += additionalLaps;
      return numLaps;
   }
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- declare the class as `private`
- include extraneous code outside the class
- include `()` with or without parameters in the class header

FINAL Unit3 FRQ - Topic order

Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares `private` instance variables that can be used to determine the lap count, the reset frequency, and the number of times the method is called

Canonical Solution:

```
public class LapTracker
{
   private int numLaps;
   private int numCalls;
   private int resetFrequency;

   public LapTracker(int resetFreq)
   {
      numLaps = 0;
      numCalls = 0;
      resetFrequency = resetFreq;
   }

   public int addLaps(int additionalLaps)
   {
      if (numCalls == resetFrequency)
      {
         numCalls = 0;
         numLaps = 0;
      }
      numCalls++;
      numLaps += additionalLaps;
      return numLaps;
   }
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

FINAL Unit3 FRQ - Topic order

- incorrectly initialize the instance variables

Responses will not earn the point if they

- declare any instance variables `static`
- omit `private` in instance variable declaration
- declare a variable outside the class
- declare an instance variable inside the constructor
- declare any of the three with a non-numeric type

Point 3

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares constructor header: `LapTracker(int ___)`

Canonical Solution:

```
public class LapTracker
{
   private int numLaps;
   private int numCalls;
   private int resetFrequency;

   public LapTracker(int resetFreq)
   {
      numLaps = 0;
      numCalls = 0;
      resetFrequency = resetFreq;
   }

   public int addLaps(int additionalLaps)
   {
      if (numCalls == resetFrequency)
      {
         numCalls = 0;
         numLaps = 0;
      }
      numCalls++;
      numLaps += additionalLaps;
      return numLaps;
   }
}
```

FINAL Unit3 FRQ - Topic order

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- declare the constructor as `private` or `static`

Point 4

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Constructor assigns correct initial value to reset frequency instance variable

Canonical Solution

```
public class LapTracker
{
   private int numLaps;
   private int numCalls;
   private int resetFrequency;

   public LapTracker(int resetFreq)
   {
      numLaps = 0;
      numCalls = 0;
      resetFrequency = resetFreq;
   }

   public int addLaps(int additionalLaps)
   {
      if (numCalls == resetFrequency)
      {
         numCalls = 0;
         numLaps = 0;
      }
      numCalls++;
      numLaps += additionalLaps;
      return numLaps;
   }
}
```

FINAL Unit3 FRQ - Topic order

0	1 ✓

The response does not earn the point.

Responses will not earn the point if they

- assign values to instance variables from an unknown source
- assign initial values to local variables instead of instance variables, even if the local variables are declared as `private`
- assign parameter to a variable with incompatible data type

Point 5

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares method header: `int addLaps(int ___)`

Canonical Solution:

```
public class LapTracker
{
   private int numLaps;
   private int numCalls;
   private int resetFrequency;

   public LapTracker(int resetFreq)
   {
      numLaps = 0;
      numCalls = 0;
      resetFrequency = resetFreq;
   }

   public int addLaps(int additionalLaps)
   {
      if (numCalls == resetFrequency)
      {
         numCalls = 0;
         numLaps = 0;
      }
      numCalls++;
      numLaps += additionalLaps;
      return numLaps;
```

FINAL Unit3 FRQ - Topic order

```
    }
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- declare method as `private`
- use incorrect method name
- use incorrect return or parameter type

Point 6

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Maintains correct lap count with reset to 0 when appropriate (*algorithm*)

Canonical Solution

```
public class LapTracker
{
   private int numLaps;
   private int numCalls;
   private int resetFrequency;

   public LapTracker(int resetFreq)
   {
      numLaps = 0;
      numCalls = 0;
      resetFrequency = resetFreq;
   }

   public int addLaps(int additionalLaps)
   {
      if (numCalls == resetFrequency)
      {
         numCalls = 0;
         numLaps = 0;
      }
```

FINAL Unit3 FRQ - Topic order

```
      numCalls++;
      numLaps += additionalLaps;
      return numLaps;
   }
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- fail to return the count
- use a variable declared outside the method as an instance variable when instance variables do not exist

Responses will not earn the point if they

- fail to declare or properly update all needed instance variables
- calculate the correct count but return something else

Point 7

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Returns calculated count from within `addLaps`

Canonical Solution:

```
public class LapTracker
{
   private int numLaps;
   private int numCalls;
   private int resetFrequency;

   public LapTracker(int resetFreq)
   {
      numLaps = 0;
      numCalls = 0;
      resetFrequency = resetFreq;
   }

   public int addLaps(int additionalLaps)
```

FINAL Unit3 FRQ - Topic order

```
   {
      if (numCalls == resetFrequency)
      {
         numCalls = 0;
         numLaps = 0;
      }
      numCalls++;
      numLaps += additionalLaps;
      return numLaps;
   }
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- return an incorrect count

Responses will not earn the point if they

- fail to return a value in some cases

FINAL Unit3 FRQ - Topic order

6. The TrashCan class, which you will write, represents a trash can.

TrashCan objects are created by calls to a constructor with a double parameter that represents the trash can's capacity, which is the maximum weight, in pounds, that a TrashCan object can hold. Assume that this value will be greater than or equal to 0. When a Trashcan object is constructed, it is initially empty and its contents have a weight of 0 pounds.

The TrashCan class contains an acceptTrash method, which has a double parameter that represents the weight, in pounds, of the trash that will be deposited into the trash can. The amount to be deposited will always be greater than 0 and less than the capacity of the can. If there is already trash in the trash can, the amount to be deposited, combined with the existing trash, may reach or exceed the can's capacity. In this case, the trash can will be filled to its capacity and then emptied before the remaining trash is accepted. The trash can should always be emptied when the weight of the trash in the can reaches the can's capacity. The acceptTrash method returns a double that represents the number of pounds of trash that can still be added to the trash can before it is completely filled.

The following table contains a sample code execution sequence and the corresponding results. The code execution sequence appears in a class other than TrashCan.

Statement	Return Value (blank if no value)	Explanation
double remaining;		
TrashCan kitchen = new TrashCan(10.0);		A TrashCan object named kitchen is constructed with a capacity of 10 pounds of trash. Initially there are 0 pounds of trash in the trash can.
remaining = kitchen.acceptTrash(2.5);	7.5	kitchen now holds 2.5 pounds of trash and can take 7.5 more pounds before it needs to be emptied.
remaining = kitchen.acceptTrash(3.5);	4.0	kitchen now holds 6 pounds of trash and can take 4 more pounds before it needs to be emptied.
remaining = kitchen.acceptTrash(6.0);	8.0	kitchen accepts 4 pounds of trash, reaching the 10-pound capacity, and is then emptied. The remaining 2 pounds of trash are then accepted. kitchen now holds 2 pounds of

FINAL Unit3 FRQ - Topic order

Statement	Return Value (blank if no value)	Explanation
		trash and can take 8 more pounds before it needs to be emptied.
TrashCan bedroom = new TrashCan(3.0);		A TrashCan object named bedroom is constructed with a capacity of 3 pounds of trash. Initially there are 0 pounds of trash in the trash can.
remaining = bedroom.acceptTrash(1.0);	2.0	bedroom now holds 1 pound of trash and can take 2 more pounds before it needs to be emptied.
remaining = bedroom.acceptTrash(3.2);	1.8	bedroom accepts 2 pounds of trash to reach the 3-pound capacity and then is emptied. The remaining 1.2 pounds of trash are then accepted. bedroom now holds 1.2 pounds of trash and can take 1.8 more pounds before it needs to be emptied.

Write the complete TrashCan class. Your implementation must meet all specifications and conform to the examples shown in the table.

Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares class header: `class TrashCan`

Canonical Solution:

```
public class TrashCan
{
    private double capacity;
    private double trashAmount;

    public TrashCan(double weightCapacity)
    {
```

FINAL Unit3 FRQ - Topic order

```
      capacity = weightCapacity;
      trashAmount = 0.0;
    }

    public double acceptTrash(double dumpWeight)
    {
      if (trashAmount + dumpWeight >= capacity)
      {
          trashAmount += dumpWeight - capacity;
      }
      else
      {
          trashAmount += dumpWeight;
      }
      return capacity – trashAmount;
    }
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- declare the class as `private`
- include extraneous code outside the class
- Include () with or without parameters in the class header

Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares `private` instance variables that can be used to determine the trash weight and capacity of the trash can

Canonical Solution:

```
public class TrashCan
{
    private double capacity;
    private double trashAmount;
```

FINAL Unit3 FRQ - Topic order

```
    public TrashCan(double weightCapacity)
    {
       capacity = weightCapacity;
       trashAmount = 0.0;
    }

    public double acceptTrash(double dumpWeight)
    {
      if (trashAmount + dumpWeight >= capacity)
      {
          trashAmount += dumpWeight - capacity;
      }
      else
      {
          trashAmount += dumpWeight;
      }
      return capacity - trashAmount;
   }
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point if they

- incorrectly initialize the instance variables

Responses will not earn the point if they

- declare any instance variable `static`
- omit `private` in instance variable declaration
- declare a variable outside the class
- declare an instance variable inside the constructor

Point 3

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares constructor header: `TrashCan(double ___)`

FINAL Unit3 FRQ - Topic order

Canonical Solution:

```
public class TrashCan
{
    private double capacity;
    private double trashAmount;

    public TrashCan(double weightCapacity)
    {
       capacity = weightCapacity;
       trashAmount = 0.0;
    }

    public double acceptTrash(double dumpWeight)
    {
      if (trashAmount + dumpWeight >= capacity)
      {
          trashAmount += dumpWeight - capacity;
      }
      else
      {
          trashAmount += dumpWeight;
      }
      return capacity - trashAmount;
   }
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- declare the constructor as `private` or `static`

Point 4

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Assigns correct initial values to capacity instance variable

FINAL Unit3 FRQ - Topic order

Canonical Solution:

```
public class TrashCan
{
    private double capacity;
    private double trashAmount;

    public TrashCan(double weightCapacity)
    {
       capacity = weightCapacity;
       trashAmount = 0.0;
    }

    public double acceptTrash(double dumpWeight)
    {
      if (trashAmount + dumpWeight >= capacity)
      {
          trashAmount += dumpWeight - capacity;
      }
      else
      {
          trashAmount += dumpWeight;
      }
      return capacity – trashAmount;
   }
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point if they

- write a class header with parameter and assign the parameter to the instance variable

Responses will not earn the point if they

- fail to declare or initialize instance variable for capacity
- assign a value to instance variables from an unknown source
- assign initial value to local variable instead of instance variable, even if the local variable is declared as `private`

Point 5

FINAL Unit3 FRQ - Topic order

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Declares method header: `double acceptTrash(double ___)`

Canonical Solution:

```
public class TrashCan
{
    private double capacity;
    private double trashAmount;

    public TrashCan(double weightCapacity)
    {
       capacity = weightCapacity;
       trashAmount = 0.0;
    }

    public double acceptTrash(double dumpWeight)
    {
      if (trashAmount + dumpWeight >= capacity)
      {
          trashAmount += dumpWeight - capacity;
      }
      else
      {
          trashAmount += dumpWeight;
      }
      return capacity - trashAmount;
   }
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- declare method as something other than `public`
- use incorrect method name
- use incorrect return or parameter type

FINAL Unit3 FRQ - Topic order

Point 6

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Correctly calculates and updates variable used to maintain amount of trash (or space remaining) in the trash can, including emptying when appropriate (*algorithm*)

Canonical Solution:

```
public class TrashCan
{
    private double capacity;
    private double trashAmount;

    public TrashCan(double weightCapacity)
    {
       capacity = weightCapacity;
       trashAmount = 0.0;
    }

    public double acceptTrash(double dumpWeight)
    {
      if (trashAmount + dumpWeight >= capacity)
      {
          trashAmount += dumpWeight - capacity;
      }
      else
      {
          trashAmount += dumpWeight;
      }
      return capacity - trashAmount;
   }
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point if they

- fail to return the correct amount

FINAL Unit3 FRQ - Topic order

- use a variable declared outside the method as an instance variable when instance variables do not exist
- declare instance variables outside the class, or in the class within a method or constructor

Responses will not earn the point if they

- fail to update the instance variable(s)
- calculate the correct remaining capacity but return something else

Point 7

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Returns calculated amount remaining from within `acceptTrash`

Canonical Solution:

```
public class TrashCan
{
    private double capacity;
    private double trashAmount;

    public TrashCan(double weightCapacity)
    {
       capacity = weightCapacity;
       trashAmount = 0.0;
    }

    public double acceptTrash(double dumpWeight)
    {
      if (trashAmount + dumpWeight >= capacity)
      {
          trashAmount += dumpWeight - capacity;
      }
      else
      {
          trashAmount += dumpWeight;
      }
      return capacity – trashAmount;
   }
}
```

FINAL Unit3 FRQ - Topic order

0	1 ✓

The response earns the point.

Responses can still earn the point if they

- return an incorrect amount

Responses will not earn the point if they

- fail to return a value in some cases

FINAL Unit3 FRQ - Topic order

7. The Apple class is used to store information about the apples to be used to bake apple pies. A partial declaration of the Apple class is shown.

```
public class Apple
{
  /**
   *  Returns the weight of the apple, in pounds
   */
  public double getWeight()
  { /* implementation not shown */ }

  /**
   *  Returns true if the apple is rotten and returns false otherwise
   */
  public boolean isRotten()
  { /* implementation not shown */ }

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

The AppleCrate class maintains a two-dimensional array of apples. A partial declaration of the AppleCrate class is shown.

```
public class AppleCrate
{
 private Apple[][] apples;

  /**
   *  Returns the number of pies that can be made from the 2D array apples
   *  if each pie requires poundsPerPie pounds of apples. If an apple is
   *  rotten, it cannot be used in a pie, nor can any adjacent apple.
   *  Preconditions: apples has at least two rows and at least two columns.
   *                 No elements in apples are null.
   */
  public int numberOfPies(double poundsPerPie)
  { /* to be implemented */}

  /* There may be instance variables, constructors,
     and methods that are not shown. */
}
```

Write the AppleCrate method numberOfPies. The method should return the total number of pies that can be made from the apples in the two-dimensional array apples if each pie requires poundsPerPie

FINAL Unit3 FRQ - Topic order

pounds of apples.

Rotten apples cannot be used in a pie, nor can any apples adjacent to a rotten apple. The apples adjacent to a given apple are the apples one element above, one element below, one element to the left of, and one element to the right of the given apple.

Suppose apples has the following contents. For each apple, the first value represents the apple's weight, in pounds, and the second value indicates whether the apple is rotten.

0.3 false	0.4 false	0.35 false	0.5 false	0.45 false	0.4 true
0.42 false	0.38 false	0.33 false	0.4 false	0.41 false	0.44 false
0.32 false	0.44 false	0.35 true	0.49 false	0.29 false	0.32 false
0.3 false	0.4 false	0.35 false	0.5 false	0.45 false	0.4 false

The shaded elements cannot be used in a pie because they represent rotten apples or apples that are adjacent to rotten apples.

For the contents of apples shown, numberOfPies(2.0) should return 3. The total weight of the apples available for pies (the non-shaded elements) is 6.14 pounds. If 2.0 pounds of apples are needed for each pie, 3 whole pies can be made, since $6.14/2.0 = 3.07$.

Complete method numberOfPies.

```
/**
 * Returns the number of pies that can be made from the 2D array apples
 * if each pie requires poundsPerPie pounds of apples. If an apple is
 * rotten, it cannot be used in a pie, nor can any adjacent apple.
 * Preconditions: apples has at least two rows and at least two columns.
 *                No elements in apples are null.
 */
public int numberOfPies(double poundsPerPie)
```

Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

FINAL Unit3 FRQ - Topic order

- Accesses all elements of `apples`

Canonical Solution:

```
public int numberOfPies(double poundsPerPie)
{
   double weight = 0;

   for (int r = 0; r < apples.length; r++)
   {
      for (int c = 0; c < apples[0].length; c++)
      {
         boolean bad = apples[r][c].isRotten();

         if (r > 0 && apples[r - 1][c].isRotten())
         {
            bad = true;
         }
         if (r < apples.length - 1 &&
             apples[r + 1][c].isRotten())
         {
            bad = true;
         }
         if (c > 0 && apples[r][c - 1].isRotten())
         {
            bad = true;
         }
         if (c < apples[0].length - 1 &&
             apples[r][c + 1].isRotten())
         {
            bad = true;
         }

         if (!bad)
         {
            weight += apples[r][c].getWeight();
         }
      }
   }
   return (int)(weight / poundsPerPie);
}
```

FINAL Unit3 FRQ - Topic order

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- access elements of `apples` out of bounds as part of adjacency checking, as long as loop bounds support traversing every element

Responses will not earn the point if they

- fail to access elements of `apples` correctly

Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Calls `isRotten` and `getWeight` on an `Apple` object

Canonical Solution:

```
public int numberOfPies(double poundsPerPie)
{
   double weight = 0;

   for (int r = 0; r < apples.length; r++)
   {
      for (int c = 0; c < apples[0].length; c++)
      {
         boolean bad = apples[r][c].isRotten();

         if (r > 0 && apples[r - 1][c].isRotten())
         {
            bad = true;
         }
         if (r < apples.length - 1 &&
             apples[r + 1][c].isRotten())
         {
            bad = true;
         }
         if (c > 0 && apples[r][c - 1].isRotten())
```

FINAL Unit3 FRQ - Topic order

```
            {
               bad = true;
            }
            if (c < apples[0].length - 1 &&
                apples[r][c + 1].isRotten())
            {
               bad = true;
            }

            if (!bad)
            {
               weight += apples[r][c].getWeight();
            }
         }
      }
      return (int)(weight / poundsPerPie);
   }
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- access elements of `apples` out of bounds
- fail to access elements of `apples` correctly

Responses will not earn the point if they

- call `isRotten` or `getWeight` incorrectly

Point 3

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Guards sum with a check for rotten apples

Canonical Solution:

```
public int numberOfPies(double poundsPerPie)
{
   double weight = 0;
```

FINAL Unit3 FRQ - Topic order

```
   for (int r = 0; r < apples.length; r++)
   {
      for (int c = 0; c < apples[0].length; c++)
      {
         boolean bad = apples[r][c].isRotten();

         if (r > 0 && apples[r - 1][c].isRotten())
         {
            bad = true;
         }
         if (r < apples.length - 1 &&
             apples[r + 1][c].isRotten())
         {
            bad = true;
         }
         if (c > 0 && apples[r][c - 1].isRotten())
         {
            bad = true;
         }
         if (c < apples[0].length - 1 &&
             apples[r][c + 1].isRotten())
         {
            bad = true;
         }

         if (!bad)
         {
            weight += apples[r][c].getWeight();
         }
      }
   }
   return (int)(weight / poundsPerPie);
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- fail to check all adjacent locations

FINAL Unit3 FRQ - Topic order

- combine the check for rotten apples with incorrect logic

Point 4

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Initializes sum variable and accumulates sum within a loop

Canonical Solution:

```
public int numberOfPies(double poundsPerPie)
{
   double weight = 0;

   for (int r = 0; r < apples.length; r++)
   {
      for (int c = 0; c < apples[0].length; c++)
      {
         boolean bad = apples[r][c].isRotten();

         if (r > 0 && apples[r - 1][c].isRotten())
         {
            bad = true;
         }
         if (r < apples.length - 1 &&
             apples[r + 1][c].isRotten())
         {
            bad = true;
         }
         if (c > 0 && apples[r][c - 1].isRotten())
         {
            bad = true;
         }
         if (c < apples[0].length - 1 &&
             apples[r][c + 1].isRotten())
         {
            bad = true;
         }

         if (!bad)
         {
            weight += apples[r][c].getWeight();
         }
```

FINAL Unit3 FRQ - Topic order

```
      }
   }
   return (int)(weight / poundsPerPie);
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- declare the sum variable as `int`

Point 5

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Identifies non-usable apples (*algorithm*)

Canonical Solution:

```
public int numberOfPies(double poundsPerPie)
{
   double weight = 0;

   for (int r = 0; r < apples.length; r++)
   {
      for (int c = 0; c < apples[0].length; c++)
      {
         boolean bad = apples[r][c].isRotten();

         if (r > 0 && apples[r - 1][c].isRotten())
         {
            bad = true;
         }
         if (r < apples.length - 1 &&
             apples[r + 1][c].isRotten())
         {
            bad = true;
         }
         if (c > 0 && apples[r][c - 1].isRotten())
```

FINAL Unit3 FRQ - Topic order

```
            {
               bad = true;
            }
            if (c < apples[0].length - 1 &&
                apples[r][c + 1].isRotten())
            {
               bad = true;
            }

            if (!bad)
            {
               weight += apples[r][c].getWeight();
            }
         }
      }
      return (int)(weight / poundsPerPie);
   }
```

0	✓ 1

The response earns the point.

Responses can still earn the point even if they

- call `isRotten` incorrectly

Responses will not earn the point if they

- fail to call `isRotten`
- fail to check current location for rot
- fail to check all adjacent locations
- combine the check for rotten apples with incorrect logic
- fail to guard access of adjacent elements to prevent bounds error

Point 6

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Returns correct integer number of pies (*algorithm*)

Canonical Solution:

FINAL Unit3 FRQ - Topic order

```
public int numberOfPies(double poundsPerPie)
{
   double weight = 0;

   for (int r = 0; r < apples.length; r++)
   {
      for (int c = 0; c < apples[0].length; c++)
      {
         boolean bad = apples[r][c].isRotten();

         if (r > 0 && apples[r - 1][c].isRotten())
         {
            bad = true;
         }
         if (r < apples.length - 1 &&
             apples[r + 1][c].isRotten())
         {
            bad = true;
         }
         if (c > 0 && apples[r][c - 1].isRotten())
         {
            bad = true;
         }
         if (c < apples[0].length - 1 &&
             apples[r][c + 1].isRotten())
         {
            bad = true;
         }

         if (!bad)
         {
            weight += apples[r][c].getWeight();
         }
      }
   }
   return (int)(weight / poundsPerPie);
}
```

0	1 ✓

The response earns the point.

FINAL Unit3 FRQ - Topic order

Responses can still earn the point even if they

- identify non-usable apples incorrectly
- call `getWeight` incorrectly

Responses will not earn the point if they

- return early
- fail to return the computed value
- fail to call `getWeight`

FINAL Unit3 FRQ - Topic order

8. The Appointment class is used to store information about a person's scheduled appointments. A partial declaration of the Appointment class is shown.

```
public class Appointment
{
  /**
   *  Returns the status of the appointment ("free", "busy", etc.)
   */
  public String getStatus()
  { /* implementation not shown */ }

  /**
   *  Returns the room number of the appointment location
   */
  public int getRoomNumber()
  { /* implementation not shown */ }

  /* There may be instance variables, constructors, and methods
     that are not shown. */
}
```

The Schedule class maintains a two-dimensional array of appointments that represents a person's schedule. A partial declaration of the Schedule class is shown.

```
public class Schedule
{
  private Appointment[][] sched;

  /**
   *  Returns the index of a column containing the fewest occurrences
   *  of the status indicated by the parameter target
   *  Preconditions: sched is not null and no elements of sched are null.
   *                 sched has at least one row and at least one column.
   */
  public int columnWithFewest(String target)
  { /* to be implemented */}

  /* There may be instance variables, constructors, and methods
     that are not shown. */
}
```

When an element of the two-dimensional array sched is accessed, the first index is used to specify the row and the second index is used to specify the column.

FINAL Unit3 FRQ - Topic order

Write the Schedule method columnWithFewest. The method should return the index of a column in sched that contains the fewest occurrences of the parameter target. If there are multiple columns that have the fewest number of occurrences of target, any of their column indices can be returned.

Suppose sched has the following contents. For each element, the first value is the appointment status and the second value is the room number.

	0	1	2	3	4
0	"free" 100	"free" 100	"free" 100	"busy" 206	"busy" 204
1	"free" 100	"free" 100	"busy" 304	"busy" 206	"busy" 202
2	"hold" 201	"busy" 105	"busy" 205	"free" 100	"busy" 205
3	"busy" 204	"free" 100	"busy" 310	"hold" 110	"free" 100
4	"busy" 204	"hold" 201	"hold" 310	"busy" 105	"free" 100
5	"busy" 105	"busy" 208	"hold" 310	"busy" 105	"free" 100

For these contents of sched, the expected behavior of columnWithFewest is as follows.

- The call columnWithFewest("busy") should return 1 because "busy" appears two times in column 1 and more than two times in each of the other columns.
- The call columnWithFewest("free") should return either 2 or 3 because "free" appears one time in column 2, one time in column 3, and more than one time in each of the other columns.
- The call columnWithFewest("hold") should return 4 because "hold" appears zero times in column 4 and one or more times in each of the other columns.

Complete method columnWithFewest.

```
/**
 * Returns the index of a column containing the fewest occurrences
 * of the status indicated by the parameter target
 * Preconditions: sched is not null and no elements of sched are null.
 *                sched has at least one row and at least one column.
 */
public int columnWithFewest(String target)
```

Point 1

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Accesses all elements of `sched` (*no bounds errors*)

FINAL Unit3 FRQ - Topic order

Canonical Solution:

```
public int columnWithFewest(String target)
{
   int minCol = 0;
   int minCount = sched.length;
   for (int col = 0; col < sched[0].length; col++)
   {
      int count = 0;
      for (int row = 0; row < sched.length; row++)
      {
         if (sched[row][col].getStatus().equals(target))
         {
            count++;
         }
      }
      if (count < minCount)
      {
         minCol = col;
         minCount = count;
      }
   }
   return minCol;
}
```

0	1 ✓

The response earns the point.

Responses will not earn the point if they

- fail to access elements of `sched` correctly

Point 2

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Calls `getStatus` to obtain a status of an `Appointment` object

Canonical Solution:

FINAL Unit3 FRQ - Topic order

```
public int columnWithFewest(String target)
{
   int minCol = 0;
   int minCount = sched.length;
   for (int col = 0; col < sched[0].length; col++)
   {
      int count = 0;
      for (int row = 0; row < sched.length; row++)
      {
         if (sched[row][col].getStatus().equals(target))
         {
            count++;
         }
      }
      if (count < minCount)
      {
         minCol = col;
         minCount = count;
      }
   }
   return minCol;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- access elements of `sched` out of bounds
- fail to access elements of `sched` correctly

Responses will not earn the point if they

- call `getStatus` incorrectly

Point 3

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Compares a status with `target`

FINAL Unit3 FRQ - Topic order

Canonical Solution:

```
public int columnWithFewest(String target)
{
   int minCol = 0;
   int minCount = sched.length;
   for (int col = 0; col < sched[0].length; col++)
   {
      int count = 0;
      for (int row = 0; row < sched.length; row++)
      {
         if (sched[row][col].getStatus().equals(target))
         {
            count++;
         }
      }
      if (count < minCount)
      {
         minCol = col;
         minCount = count;
      }
   }
   return minCol;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- access a status incorrectly

Responses will not earn the point if they

- use `==` to compare statuses

Point 4

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

FINAL Unit3 FRQ - Topic order

- Compares identified counts of target value for two columns

Canonical Solution:

```
public int columnWithFewest(String target)
{
   int minCol = 0;
   int minCount = sched.length;
   for (int col = 0; col < sched[0].length; col++)
   {
      int count = 0;
      for (int row = 0; row < sched.length; row++)
      {
         if (sched[row][col].getStatus().equals(target))
         {
            count++;
         }
      }
      if (count < minCount)
      {
         minCol = col;
         minCount = count;
      }
   }
   return minCol;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- determine the number of elements in each row instead of each column
- compute an incorrect count
- reverse an inequality when comparing counts

Responses will not earn the point if they

- fail to maintain the count of a previous column to compare to the current column

Point 5

FINAL Unit3 FRQ - Topic order

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Determines the number of elements in each accessed column that match target value (*algorithm*)

Canonical Solution:

```
public int columnWithFewest(String target)
{
   int minCol = 0;
   int minCount = sched.length;
   for (int col = 0; col < sched[0].length; col++)
   {
      int count = 0;
      for (int row = 0; row < sched.length; row++)
      {
         if (sched[row][col].getStatus().equals(target))
         {
            count++;
         }
      }
      if (count < minCount)
      {
         minCol = col;
         minCount = count;
      }
   }
   return minCol;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- determine the number of elements in each row instead of each column

Responses will not earn the point if they

- fail to reset the count when advancing to the next column
- fail to initialize a count

FINAL Unit3 FRQ - Topic order

Point 6

Select a point value to view scoring criteria or to score the response.

To earn this point, responses must demonstrate the following:

- Returns index of column with smallest computed count (*algorithm*)

Canonical Solution:

```
public int columnWithFewest(String target)
{
   int minCol = 0;
   int minCount = sched.length;
   for (int col = 0; col < sched[0].length; col++)
   {
      int count = 0;
      for (int row = 0; row < sched.length; row++)
      {
         if (sched[row][col].getStatus().equals(target))
         {
            count++;
         }
      }
      if (count < minCount)
      {
         minCol = col;
         minCount = count;
      }
   }
   return minCol;
}
```

0	1 ✓

The response earns the point.

Responses can still earn the point even if they

- compute the count for individual columns incorrectly

Responses will not earn the point if they

FINAL Unit3 FRQ - Topic order

- confuse rows with columns
- fail to initialize the minimum count
- fail to initialize the index
- fail to return the computed value
- return the count instead of the index
- reverse an inequality when comparing counts

FINAL Unit3 FRQ - Topic order

9. SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.

Assume that the classes listed in the Java Quick Reference have been imported where appropriate. Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

This question involves a scoreboard for a game. The game is played between two teams who alternate turns so that at any given time, one team is active and the other team is inactive. During a turn, a team makes one or more plays. Each play can score one or more points and the team's turn continues, or the play can fail, in which case no points are scored and the team's turn ends. The `Scoreboard` class, which you will write, is used to keep track of the score in a game.

The `Scoreboard` class contains a constructor and two methods.

The constructor has two parameters. The first parameter is a `String` containing the name of team 1, and the second parameter is a `String` containing the name of team 2. The game always begins with team 1 as the active team.

The `recordPlay` method has a single nonnegative integer parameter that is equal to the number of points scored on a play or `0` if the play failed. If the play results in one or more points scored, the active team's score is updated and that team remains active. If the value of the parameter is `0`, the active team's turn ends and the inactive team becomes the active team. The `recordPlay` method does not return a value.

The `getScore` method has no parameters. The method returns a `String` containing information about the current state of the game. The returned string begins with the score of team 1, followed by a hyphen (`"-"`), followed by the score of team 2, followed by a hyphen, followed by the name of the team that is currently active.

The following table contains a sample code execution sequence and the corresponding results. The code execution sequence appears in a class other than `Scoreboard`.

Statement	Value Returned (blank if none)	Explanation
`String info;`		
`Scoreboard game = new Scoreboard("Red", "Blue");`		`game` is a new `Scoreboard` for a game played between team 1, whose name is`"Red"`, and team 2, whose name is `"Blue"`. The active team is set to team 1.
`info = game.getScore();`	`"0-0-Red"`	
`game.recordPlay(1);`		Team 1 earns 1 point because the game always begins with team 1 as the active team.

FINAL Unit3 FRQ - Topic order

Statement	Value Returned (blank if none)	Explanation
`info = game.getScore();`	`"1-0-Red"`	
`game.recordPlay(0);`		Team 1's play failed, so team 2 is now active.
`info = game.getScore();`	`"1-0-Blue"`	
`info = game.getScore();`	`"1-0-Blue"`	The score and state of the game are unchanged since the last call to `getScore`.
`game.recordPlay(3);`		Team 2 earns 3 points.
`info = game.getScore();`	`"1-3-Blue"`	
`game.recordPlay(1);`		Team 2 earns 1 point.
`game.recordPlay(0);`		Team 2's play failed, so team 1 is now active.
`info = game.getScore();`	`"1-4-Red"`	
`game.recordPlay(0);`		Team 1's play failed, so team 2 is now active.
`game.recordPlay(4);`		Team 2 earns 4 points.
`game.recordPlay(0);`		Team 2's play failed, so team 1 is now active.
`info = game.getScore();`	`"1-8-Red"`	
`Scoreboard match = new Scoreboard("Lions", "Tigers");`		`match` is a new and independent `Scoreboard` object.
`info = match.getScore();`	`"0-0-Lions"`	
`info = game.getScore();`	`"1-8-Red"`	

Write the complete `Scoreboard` class. Your implementation must meet all specifications and conform to the examples shown in the preceding table.

Scoreboard (9 points)

FINAL Unit3 FRQ - Topic order

Select a point value to view scoring criteria, solutions, and/or examples and to score the response. +1 indicates a point earned.

+1 **[Skill 3.B]** Declares class header: `class Scoreboard`

Responses will not earn the point if they

- declare the class as something other than `public`

+1 **[Skill 3.B]** Declares at least one `private String` instance variable and one `private int` instance variable

Responses will not earn the point if they

- declare any instance variable `static`
- declare a variable outside the class

+1 **[Skill 3.B]** Declares constructor header: `Scoreboard (String ___, String ___)` and constructor initializes both team name instance variables using parameters

Responses can still earn the point even if they

- declare instance variable(s) outside the class, or in the class within a method or constructor

Responses will not earn the point if they

- fail to declare or initialize instance variables for both team names
- declare the constructor as something other than `public`

+1 **[Skill 3.B]** Declares method headers: `public void recordPlay (int ___)` `public String getScore()`

Responses will not earn the point if they

- use incorrect method names
- omit or declare incorrectly either method header
- omit `public` in either method header or declare either method as something other than `public`

+1 **[Skill 3.C]** Recording method checks for parameter value of zero

Responses can still earn the point even if they

- use a method name inconsistent with the examples, as long as it is recognizably equivalent

+1 **[Skill 3.B]** Recording method increases at least one declared instance variable representing one team's score

Responses can still earn the point even if they

FINAL Unit3 FRQ - Topic order

- declare any instance variable incorrectly, outside the class, or in the class within a method or constructor
- use something other than the parameter to update the instance variable
- use a method name inconsistent with the examples, as long as it is recognizably equivalent

+1 **[Skill 3.C]** Recording method switches active team

Responses can still earn the point even if they

- perform the switch in a method other than the recording method
- store the switched active team in a local variable, as long as the switch occurs in both active team cases
- use a method name inconsistent with the examples, as long as it is recognizably equivalent
- perform the switch when the parameter is not zero

+1 **[Skill 3.C]** Recording method adds correct number of points to the active team's score (*algorithm*)

Responses can still earn the point even if they

- fail to switch active team correctly
- declare an instance variable that holds a team's score outside the class, or in the class within a method or constructor
- use a method name inconsistent with the examples, as long as it is recognizably equivalent

Responses will not earn the point if they

- switch teams when the parameter is positive
- fail to declare an instance variable to track the active team, initialize it incorrectly, or never change its value
- add correct number of points for only one team
- increase score by something other than the parameter
- fail to declare instance variables to hold both teams' scores

+1 **[Skill 3.C]** Accessor method builds and returns specified string (*algorithm*)

Responses can still earn the point even if they

- fail to declare instance variables and use variables from constructor or methods within the class
- use a method name inconsistent with the examples, as long as it is recognizably equivalent

Responses will not earn the point if they

FINAL Unit3 FRQ - Topic order

- omit the literal hyphens in the constructed string

Canonical Solution:

```
public class Scoreboard
{
   private String team1Name, team2Name;
   private int whoseTurn;
   private int score1, score2;

   public Scoreboard(String team1, String team2)
   {
      team1Name = team1;
      team2Name = team2;
      whoseTurn = 1;
      score1 = 0;
      score2 = 0;
   }

   public void recordPlay(int points)
   {
      if (points == 0)
      {
         if (whoseTurn == 1)
         {
            whoseTurn = 2;
         }
         else
         {
            whoseTurn = 1;
         }
      }
      else
      {
         if (whoseTurn == 1)
         {
            score1 += points;
         }
         else
         {
            score2 += points;
         }
      }
   }

   public String getScore()
   {
      String result = score1 + "-" + score2 + "-";
      if (whoseTurn == 1)
      {
         result += team1Name;
      }
      else
      {
         result += team2Name;
      }
      return result;
   }
}
```

FINAL Unit3 FRQ - Topic order

0	1	2	3	4	5	6	7	8	9 ✓

Total number of points earned is equal to 9.

- ☐ +1 Declares class header: `class Scoreboard` **(Points Earned)**
- ☐ +1 Declares at least one `private String` instance variable and one `private int` instance variable **(Points Earned)**
- ☐ +1 Declares constructor header: `Scoreboard (String ___, String ___)` and constructor initializes both team name instance variables using parameters **(Points Earned)**
- ☐ +1 Declares method headers: `public void recordPlay (int ___)` `public String getScore()` **(Points Earned)**
- ☐ +1 Recording method checks for parameter value of zero **(Points Earned)**
- ☐ +1 Recording method increases at least one declared instance variable representing one team's score **(Points Earned)**
- ☐ +1 Recording method switches active team **(Points Earned)**
- ☐ +1 Recording method adds correct number of points to the active team's score (*algorithm*) **(Points Earned)**
- ☐ +1 Accessor method builds and returns specified string (*algorithm*) **(Points Earned)**

Canonical Solution:

FINAL Unit3 FRQ - Topic order

```
public class Scoreboard
{
   private String team1Name, team2Name;
   private int whoseTurn;
   private int score1, score2;

   public Scoreboard(String team1, String team2)
   {
      team1Name = team1;
      team2Name = team2;
      whoseTurn = 1;
      score1 = 0;
      score2 = 0;
   }

   public void recordPlay(int points)
   {
      if (points == 0)
      {
         if (whoseTurn == 1)
         {
            whoseTurn = 2;
         }
         else
         {
            whoseTurn = 1;
         }
      }
      else
      {
         if (whoseTurn == 1)
         {
            score1 += points;
         }
         else
         {
            score2 += points;
         }
      }
   }

   public String getScore()
   {
      String result = score1 + "-" + score2 + "-";
      if (whoseTurn == 1)
      {
         result += team1Name;
      }
      else
      {
         result += team2Name;
      }
      return result;
   }
}
```

FINAL Unit3 FRQ - Topic order

10. SHOW ALL YOUR WORK. REMEMBER THAT PROGRAM SEGMENTS ARE TO BE WRITTEN IN JAVA.

Assume that the classes listed in the Java Quick Reference have been imported where appropriate. Unless otherwise noted in the question, assume that parameters in method calls are not `null` and that methods are called only when their preconditions are satisfied.
In writing solutions for each question, you may use any of the accessible methods that are listed in classes defined in that question. Writing significant amounts of code that can be replaced by a call to one of these methods will not receive full credit.

This question involves the implementation of the `AddtionPattern` class, which generates a number pattern. The `AdditionPattern` object is constructed with two positive integer parameters, as described below.

The first positive integer parameter indicates the starting number in the pattern.
The second positive integer parameter indicates the value that is to be added to obtain each subsequent number in the pattern.

The `AdditionPattern` class also supports the following methods.

`currentNumber`, which returns the current number in the pattern
`next`, which moves to the next number in the pattern
`prev`, which moves to the previous number in the pattern or takes no action if there is no previous number

The following table illustrates the behavior of an `AdditionPattern` object that is instantiated by the following statement.

```
AdditionPattern plus3 = new AdditionPattern(2, 3);
```

Method Call	Value Returned (blank if no value)	Explanation
`plus3.currentNumber();`	`2`	The current number is initially the starting number in the pattern.
`plus3.next();`		The pattern adds `3` each time, so move to the next number in the pattern (`5`).
`plus3.currentNumber();`	`5`	The current number is `5`.
`plus3.next();`		The pattern adds `3` each time, so move to the next number in the pattern (`8`).
`plus3.next();`		The pattern adds `3` each time, so move to the next number in the pattern (`11`).
`plus3.currentNumber();`	`11`	The current number is `11`.

FINAL Unit3 FRQ - Topic order

`plus3.prev();`		Move to the previous number in the pattern (`8`).
`plus3.prev();`		Move to the previous number in the pattern (`5`).
`plus3.prev();`		Move to the previous number in the pattern (`2`).
`plus3.currentNumber();`	`2`	The current number is `2`.
`plus3.prev();`		There is no previous number in the pattern prior to `2`, so no action is taken.
`plus3.currentNumber();`	`2`	The current number is `2`.

Write the complete `AditonPattern` class. Your implementation must meet all specifications and conform to all examples.

AdditionPattern class (9 points)

Points earned:

+1 [Skill 3.B] Correct class header (must not be private)

+1 [Skill 3.B] Declares appropriate instance variables (must be private)

+1 [Skill 3.B] Correct constructor header (must not be private)

+1 [Skill 3.B] Constructor correctly initializes instance variables

Response earns this point, but incurs general penalty z below if it...

· returns a value from the constructor

Note that general penalty z can only be incurred once, even if the error is made multiple times.

+1 [Skill 3.B] Correct method headers for next and prev

+1 [Skill 3.B] Correct method header for currentNumber

+1 [Skill 3.B] Correct implementation of currentNumber

+1 [Skill 3.B] next and prev each update state by the appropriate amount

Response earns this point, but incurs general penalty z below if it...

· returns a value from either method

Note that general penalty z can only be incurred once, even if the error is made multiple times.

FINAL Unit3 FRQ - Topic order

+1 [Skill 3.C] prev prevents moving to a number less than the start value

Response earns this point, but incurs general penalty z below if it...

· returns a value from the method

Note that general penalty z can only be incurred once, even if the error is made multiple times.

General Penalties:

-1 (w) Extraneous code that causes side-effect (e.g., printing to output, incorrect precondition check)

-1 (x) Local variables used but none declared

-1 (z) Void method or constructor that returns a value

Canonical Solution:

FINAL Unit3 FRQ - Topic order

```
public class AdditionPattern
{
   private int start;
   private int increase;
   private int current;

   public AdditionPattern(int s, int i)
   {
      start = s;
      increase = i;
      current = start;
   }

   public int currentNumber()
   {
      return current;
   }

   public void next()
   {
      current += increase;
   }

   public void prev()
   {
      if (current - increase >= start)
      {
         current -= increase;
      }
   }

}
```

Alternate canonical solution:

FINAL Unit3 FRQ - Topic order

```
public class AdditionPattern
{
   private int x;
   private int start;
   private int increase;

   public AdditionPattern(int s, int i)
   {
      start = s;
      increase = i;
      x = 0;
   }

   public int currentNumber()
   {
      return increase * x + start;
   }

   public void next()
   {
      x++;
   }

   public void prev()
   {
      if (x - 1 >= 0)
      {
         x--;
      }
   }

}
```

✓

0	1	2	3	4	5	6	7	8	9

Total number of points earned (minus penalties) is equal to 9.

- ☐ **+1** Correct class header (must not be private) (**Points earned**)
- ☐ **+1** Declares appropriate instance variables (must be private) (**Points earned**)
- ☐ **+1** Correct constructor header (must not be private) (**Points earned**)
- ☐ **+1** Constructor correctly initializes instance variables (**Points earned**)
- ☐ **+1** Correct method headers for next and prev (**Points earned**)
- ☐ **+1** Correct method header for currentNumber (**Points earned**)

FINAL Unit3 FRQ - Topic order

- [ ] **+1** Correct implementation of currentNumber (**Points earned**)
- [ ] **+1** next and prev each update state by the appropriate amount (**Points earned**)
- [ ] **+1** prev prevents moving to a number less than the start value (**Points earned**)
- [ ] -1 (w) Extraneous code that causes side-effect (e.g., printing to output, incorrect precondition check) **(General Penalties)**
- [ ] -1 (x) Local variables used but none declared **(General Penalties)**
- [ ] -1 (z) Void method or constructor that returns a value **(General Penalties)**

Canonical Solution:

```
public class AdditionPattern
{
   private int start;
   private int increase;
   private int current;

   public AdditionPattern(int s, int i)
   {
      start = s;
      increase = i;
      current = start;
   }

   public int currentNumber()
   {
      return current;
   }

   public void next()
   {
      current += increase;
   }

   public void prev()
   {
      if (current - increase >= start)
      {
         current -= increase;
      }
   }

}
```

Alternate canonical solution:

FINAL Unit3 FRQ - Topic order

```
public class AdditionPattern
{
   private int x;
   private int start;
   private int increase;

   public AdditionPattern(int s, int i)
   {
      start = s;
      increase = i;
      x = 0;
   }

   public int currentNumber()
   {
      return increase * x + start;
   }

   public void next()
   {
      x++;
   }

   public void prev()
   {
      if (x - 1 >= 0)
      {
         x--;
      }
   }

}
```